EPIC-E — Case Discussions with De-identification Enforcement — Technical Spec

Status: Draft — for stakeholder review **Date**: 14 July 2026 **Author**: Adrian Hall (Technical Lead), drafted with Claude Code **Scope**: The fifth per-epic technical spec. Builds directly on EPIC-C's forum spec (`docs/superpowers/specs/2026-07-14-epic-c-forum-technical-spec.md`) — a case discussion is a `community.posts` row with `type = case_discussion` — and on the architecture spec's Section 4.3 (`identity.case_attestations`, "the deliberate exception" where identity and community data are joined). Read both first.

Source of truth: `docs/askapeer-prd-v0.1.md`, Section 6.2 (template fields) and Section 10 (Case Discussions & Patient Safety Policy) in full — this epic exists entirely to implement Section 10's policy, which the PRD itself calls out as simultaneously "among the highest-value features of the platform and the highest-risk."

Contents

1. Scope
2. Data model
3. Publish flow
4. The de-identification checklist
5. The attestation
6. API endpoints
7. Disclaimer and priority reporting
8. Post-publish editing
9. Cross-epic dependencies introduced by this spec
10. Non-functional notes specific to EPIC-E
11. Test plan
12. Open questions

---

1. Scope

In scope: the structured case-discussion template (PRD Section 6.2), the mandatory de-identification checklist and attestation gate (PRD Section 10.2–10.3) that must complete before a case discussion becomes visible to any other member, the platform disclaimer (Section 10.5), and the priority-reporting category this content type requires (Section 10.4, completed by EPIC-F).

Out of scope: general forum mechanics (posting, commenting, tagging, search — EPIC-C, which this epic's published case discussions become ordinary participants in), the moderation queue and action types themselves (EPIC-F), kudos on case discussion answers (EPIC-D, unchanged for this post type).

---

2. Data model

A case discussion is a `community.posts` row (EPIC-C's table) with `type = case_discussion`, extended with a 1:1 structured-fields table — the PRD's Section 6.2 template is nine distinct structured questions, not a single free-text body, so a dedicated table is more faithful to the requirement than overloading EPIC-C's generic `title`/`body` columns:

```
community.case_details
  post_id                   uuid PK, FK -> community.posts
  presenting_complaint      text
  relevant_history          text      -- "non-identifying" per
template item 2;
                                       -- enforcement is the
checklist (Section 4),
                                       -- not a structural
constraint on this field
  subjective_findings       text
  objective_findings        text
  red_flags_considered      text
  differential_diagnosis    text
  interventions_tried       text
  response_to_treatment     text
  community_question        text      -- "specific question to
the community"
```

The attestation itself lives in `identity.case_attestations` (architecture spec, Section 4.3) — reproduced here for reference, not redefined:

```
identity.case_attestations
  id                uuid PK
  member_id         uuid FK -> identity.members
  post_id           uuid FK -> community.posts
  attestation_text  text
  checklist_snapshot jsonb          -- the exact checklist state
at the moment of attestation
  attested_at       timestamptz
  ip_address        inet
```

`checklist_snapshot` captures the Section 4 checklist as it stood *at the moment of attestation* — not a live reference to some other mutable checklist state — so the audit record is self-contained even if the checklist's own wording changes in a future policy update.

---

3. Publish flow

```
1. Draft creation
   POST /v1/case-discussions  { category_id, tag_ids[], <9
template fields> }
   -> community.posts (type=case_discussion, status=draft),
community.case_details row
   -- "draft" is a status value this epic requires EPIC-C's
posts.status enum to gain;
      see Section 9 — EPIC-C's spec, already written, only has
(published, removed).

2. Editing
   PATCH /v1/case-discussions/:post_id     -- freely editable
while status=draft

3. Checklist completion
   PUT /v1/case-discussions/:post_id/checklist   { 8 boolean
items, Section 4 }
   -> all 8 must be true before step 4 is reachable

4. Attestation and publish
   POST /v1/case-discussions/:post_id/attest     { confirms the
Section 5 attestation text }
   -> requires: all checklist items true (server-side re-check,
not just a client gate)
   -> writes identity.case_attestations (member_id resolved
server-side from the
      caller's session — never supplied by the client)
   -> flips community.posts.status: draft -> published, in the
same transaction
      as the attestation write
```

The checklist and attestation are two distinct steps rather than one combined submit, so the UI can show the checklist as a genuine gate (each item individually checkable, reviewable before commitment) with the attestation as a separate, deliberate final action — matching the PRD's own framing of the attestation as something recorded "with timestamp" as a specific event, not incidental to checking eight boxes.

---

4. The de-identification checklist

The eight items are exactly PRD Section 10.2's list — this spec doesn't add or remove any:

1. No patient names, initials, or aliases
2. No address, postcode, or identifying location data
3. No exact date of birth — age expressed as a band (e.g. 40–49 years)
4. No exact treatment dates — timelines expressed as relative (e.g. "3 weeks post-injury")
5. No facility, club, or team name that would uniquely identify the patient
6. No patient photographs showing faces, unique tattoos, scars, or identifying features

7. Images reviewed for embedded metadata — platform strips EXIF automatically; content compliance confirmed
8. No uploaded documents contain patient identifiers

Enforcement is attestation-based, not structurally validated, for items 1, 2, 5, and 8 — the platform cannot algorithmically detect a patient name or a uniquely-identifying facility name in free text (the same limitation the EPIC-B spec notes for handle names attempting to encode a real identity). Items 3 and 4 are different: these are candidates for **structural enforcement**, not just a checkbox — e.g. an age-band selector (a fixed set of bands, not a free date-of-birth field) and a relative-timeline input (a "X weeks/months post-injury" structured field) in the `community.case_details` template itself, rather than trusting an author to remember to phrase a free-text field as a band. This spec proposes structural enforcement for items 3/4 as a meaningfully stronger safeguard than the PRD's own wording implies (which frames all eight as checklist items) — flagged as a recommendation for confirmation in Section 12, not assumed authorised unilaterally.

Items 6 and 7 depend on **image attachment support existing at all**, which is currently a Could-have in EPIC-C's spec (Section 7 there), not a committed MVP feature — a real gap between what this checklist assumes exists and what's confirmed to ship. Flagged prominently in Section 12.

---

5. The attestation

Exact wording, PRD Section 10.3:

> *"I confirm that this case discussion is de-identified in accordance with Askapeer's patient privacy policy. I understand that any breach of patient confidentiality is a serious professional and legal matter and may result in permanent removal from the platform and referral to my professional regulatory body."*

The API stores this exact text in `attestation_text` (not just a boolean "attested = true") — if the wording is ever revised, historical attestations remain an accurate record of what the member actually agreed to at the time, consistent with the architecture spec's general audit-immutability philosophy (Section 4.4). `ip_address` is captured server-side from the request, per the PRD's "recorded with timestamp and linked to the member's verified identity" requirement — this is exactly the scenario the architecture spec's Section 4.3 "deliberate exception" describes, so no new reasoning is needed here beyond pointing at it.

---

6. API endpoints

Already laid out in Section 3's flow; reproduced as a flat list for reference:

```
POST /v1/case-discussions
PATCH /v1/case-discussions/:post_id                -- draft only
PUT   /v1/case-discussions/:post_id/checklist
POST  /v1/case-discussions/:post_id/attest
GET   /v1/case-discussions/:post_id                -- published
view; renders the Section 7
```

```
disclaimer; never exposes attestation_text,

ip_address, or member_id — those stay behind

IdentityService per the architecture spec's
                                                    access
rules, same as everywhere else
```

A published case discussion is otherwise an ordinary EPIC-C post from that point forward — commentable, taggable, searchable, kudos-able (EPIC-D) — no separate rendering path beyond the disclaimer banner and the structured-field layout instead of a free body.

---

7. Disclaimer and priority reporting

Every case-discussion page carries the exact disclaimer text from PRD Section 10.5:

> *"This discussion is for peer learning purposes only. Responses represent individual practitioner perspectives and do not constitute clinical advice. Practitioners remain responsible for applying professional judgement appropriate to their individual clinical context and local scope of practice."*

This is static content rendered by the client on any `type = case_discussion` post — no data-model weight.

Priority reporting (PRD Section 10.4): the report category `identifiable_patient_information` (already present in the architecture spec's `community.reports.category` enum, Section 4.2, with `priority boolean generated from category`) must be selectable when reporting *any* case discussion — this epic's responsibility is only to confirm that category exists and is offered on this content type; the queue ordering and moderator actions themselves belong to EPIC-F.

---

8. Post-publish editing

Proposed: published case discussions are not author-editable. The attestation (Section 5) is tied to a specific `checklist_snapshot` at a specific moment — allowing free edits after publish would let the actual content diverge from what was attested to, silently invalidating the record's meaning. If a correction is genuinely needed (PRD Section 10.4 names "request a corrected resubmission" as a moderation action), this spec proposes that goes through EPIC-F as a moderator-initiated action, likely requiring the case to be unpublished and re-attested against a fresh checklist rather than a live in-place edit. This is this spec's own design position, not a PRD requirement, and is flagged in Section 12 alongside the specific new moderation-action type it implies for EPIC-F.

---

9. Cross-epic dependencies introduced by this spec

This epic requires two things of already-written specs that weren't anticipated when those specs were drafted — surfaced explicitly rather than silently patched into a committed document:

- **EPIC-C's** `community.posts.status` **enum** (currently `published`, `removed`) needs a `draft` value added to support the multi-step publish flow in Section 3. Ordinary questions/answers don't need a draft state (EPIC-C's spec assumes immediate publish on creation); case discussions do.
- **EPIC-F's moderation-action-type enum** (currently `remove_content`, `warn`, `suspend`, `expel`, per the architecture spec, Section 7.2) needs a `request_correction` action to match PRD Section 10.4's "request a corrected resubmission" — not yet written since EPIC-F's spec doesn't exist yet, but flagged here so it's not missed when it is.

---

10. Non-functional notes specific to EPIC-E

- **The attestation write is the one case in this whole epic that crosses into** `identity`**-schema territory** from an otherwise community-facing flow — exactly the exception the architecture spec documents (Section 4.3). The `POST .../attest` endpoint's implementation needs the same care as `IdentityService`'s own endpoints: it must resolve `member_id` server-side from the session, never accept it from the client, and never echo it back in the response.
- **EXIF stripping** (checklist item 7) is already specified generically at the architecture level (Section 3, S3 worker) — this epic doesn't need its own mechanism, only to confirm image upload exists at all for case discussions specifically (Section 4's flagged gap).

---

11. Test plan

- **Checklist gate**: `POST .../attest` is rejected server-side if any checklist item is false, even if the client's own UI would have blocked submission — the server must not trust client-side gating alone.
- **Attestation immutability**: `identity.case_attestations` rows cannot be updated or deleted (INSERT-only grant, same mechanism as `identity.verification_decisions` in the EPIC-A spec).
- **Draft privacy**: a `draft` case discussion is not visible via any endpoint to any handle other than its author (and moderators, if EPIC-F needs draft visibility for some reason — flagged in Section 12) — it must not leak into search, feeds, or another member's `GET /v1/posts` results before publish.
- **Post-publish edit lock**: `PATCH /v1/case-discussions/:post_id` is rejected once `status = published`.
- **Disclaimer rendering**: every case-discussion page includes the exact Section 7 text.
- **Age-band/relative-date structural fields** (if the Section 4 recommendation is adopted): a free-text date of birth or absolute

treatment date cannot be submitted in fields 3/4 at all, not merely discouraged by the checklist.

---

12. Open questions

- **Structural enforcement of checklist items 3/4** (Section 4): should date-of-birth and treatment-date fields be structurally restricted to bands/relative-time rather than relying purely on the attestation checkbox? This spec recommends yes; needs sign-off since it's a stronger (and more UI-involved) safeguard than the PRD's own wording strictly requires.
- **Image attachment dependency** (Section 4): checklist items 6/7 assume case discussions can carry images, but EPIC-C currently lists image attachments as Could-have, not committed MVP scope. If case discussions need images to be genuinely usable for clinical case sharing, this may need to become Must-have specifically for this epic even if general forum image attachments stay optional — needs a scope decision.
- **EPIC-C schema amendment** (Section 9): adding `draft` to `posts.status` — a small change, but EPIC-C's spec is already committed, so this should be reconciled explicitly (either an amendment to that spec or treated as EPIC-E-specific extension) rather than assumed.
- **EPIC-F action-type addition** (Section 9): `request_correction` needs to land in EPIC-F's spec when it's written; flagged so it isn't missed.
- **Corrected resubmission mechanics** (Section 8): does a "corrected resubmission" create a new post, or unpublish/re-edit/re-attest the same one? The PRD names the action but not the mechanics — this spec's proposal (unpublish, re-attest) is a starting point, not a decision.
- **Draft visibility to moderators**: should an in-progress (unattested) draft ever be visible to moderation for any reason (e.g. a safety escalation before publish)? Not addressed by the PRD; likely no given nothing has been published yet, but worth an explicit answer rather than an assumption.